Lab 09 — Praktijklab: de stack in één bestand

Doel: de volledige stack `web → api → db` starten met één bestand, de startvolgorde zien die de healthchecks opleggen, de stack dagelijks uitbaten (`logs`, `exec`, `stop`, `down`), schakelen tussen ontwikkeling en productie met override-bestanden, en de grenzen van podman-compose aanraken.

Vereisten — Labs 01 tot 08 afgewerkt. Image `api:1.0` aanwezig, `~/labo-docker/05/Dockerfile` aanwezig. **podman-compose geïnstalleerd**: `sudo apt install podman-compose` (is de versie die `podman compose version` toont lager dan 1.1, kies dan `pipx install podman-compose`). De uitvoer hieronder komt van podman-compose 1.6.0.

Meegeleverde bestanden (`files/`) - `compose.yaml` — de stack: `db` (PostgreSQL, volume, secret, healthcheck), `api` (image of `build: ../05`, `env_file`, `depends_on`), `web` (nginx, poort 18080). - `compose.override.yaml` — toevoegingen voor ontwikkeling, automatisch gelezen: database gepubliceerd op `127.0.0.1:5432`, profiel `dev`. - `compose.prod.yaml` — productievariant, te combineren met `-f`: `restart: unless-stopped`, profiel `prod`. - `dot-env` — te hernoemen naar `.env`: `API_TAG=1.0`, gesubstitueerd in `compose.yaml`. - `api.env` — de configuratie van de API (lab 08). - `nginx-proxy.conf`, `index.html` — de frontend en de reverse proxy van lab 07.

Stap 0 — Opzet

```
mkdir -p ~/labo-docker/09 && cd ~/labo-docker/09
cp <pad-naar-het-lab>/files/* . && mv dot-env .env
printf 'S3cr3t-pg' > db_password.txt
podman compose version 2>&1 | tail -2
podman compose config | grep -E '^name:|image:|context:'
```

Observeer de banner `>>>> Executing external compose provider "/usr/bin/podman-compose"` en dan `podman-compose version 1.6.0`; en in de samengevoegde configuratie: `name: stack`, `image: localhost/api:1.0` (de `${API_TAG:-1.0}` is gesubstitueerd), `context: /home/<jij>/labo-docker/05`.

Uitleg. `podman compose` geeft door aan podman-compose. `config` toont wat de tool van de YAML begrepen heeft, override inbegrepen — te lezen vóór elke `up`.

Stap 1 — `up`, en de startvolgorde

```
podman compose up -d
podman compose ps
```

Observeer de drie containers aangemaakt in de volgorde `stack_db_1`, `stack_api_1`, `stack_web_1`, en in `ps` verschoven leeftijden: `db Up 11 seconds (healthy)`, `api Up 6 seconds (healthy)`, `web Up 1 second`. Elke dienst heeft gewacht tot de vorige `healthy` was.

```
podman ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
podman network ls ; podman volume ls ; podman pod ls
```

Observeer `127.0.0.1:5432->5432/tcp` op `db` (de ontwikkeloverride is toegepast), `0.0.0.0:18080->80/tcp` op `web` ; een netwerk `stack_default` , een volume `stack_pgdata` , en een pod `pod_stack` met `3` containers **zonder** infra.

Uitleg. Alles draagt het voorvoegsel van het project. De pod is een groepering eigen aan podman-compose: elke dienst houdt zijn eigen IP en DNS-naam.

Stap 2 — De stack uitbaten

```
curl -s localhost:18080/api/actuator/health ; echo ; curl -s localhost:18080/api/ ; echo
podman compose logs --tail=2 api
podman compose exec db psql -U postgres -c "CREATE TABLE clients(id serial, nom text); INSERT
INTO clients(nom) VALUES ('Durand');"
podman compose exec api env | grep -E 'APP_|DB_'
podman run --rm --network stack_default alpine getent hosts api db web
```

Observeer `{"status":"UP"}` en `{"message":"bonjour depuis compose","profile":"dev"}` (het bericht komt uit `api.env` , het profiel uit de override); logs voorafgegaan door `[api] |` ; `CREATE TABLE` / `INSERT 0 1` ; de variabelen `DB_HOST=db` , `DB_PASSWORD_FILE=/run/secrets/db_password` ; en de drie namen geresolved naar `10.89.0.x` .

Uitleg. `podman compose exec` en `podman compose logs` richten zich op een **dienst** via zijn naam; daaronder zijn het `podman exec stack_db_1` en `podman logs stack_api_1` .

Stap 3 — Stoppen, herstarten, en de `502` terugvinden

```
podman inspect stack_api_1 --format '{{.NetworkSettings.Networks.stack_default.IPAddress}}'
podman compose stop api
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman compose start api ; sleep 6
podman inspect stack_api_1 --format '{{.NetworkSettings.Networks.stack_default.IPAddress}}'
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman compose restart web ; sleep 2
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
```

Observeer `10.89.0.3` , daarna `502` (of `504` , naargelang het moment) terwijl de API gestopt is, daarna een **ander** adres na `start` (`10.89.0.6`) en nog altijd `502` : nginx houdt het oude IP bij (lab 07). Na `restart web` : `200` .

Uitleg. Compose verandert daar niets aan: een `stop` / `start` geeft het adres vrij en kent een nieuw toe, en nginx resolvet alleen bij het opstarten.

Stap 4 — `down` bewaart de gegevens, `down -v` niet

```
podman compose down
podman ps -a --format '{{.Names}}' ; podman network ls --format '{{.Name}}' ; podman pod ls --
format '{{.Name}}' ; podman volume ls --format '{{.Name}}'
podman compose up -d ; sleep 12
podman compose exec db psql -U postgres -c "SELECT * FROM clients;"
```

Observeer na `down`: geen enkele container, alleen het netwerk `podman`, geen pod — maar `stack_pgdata` **nog altijd aanwezig**. Na de tweede `up` zijn de tabel `clients` en `Durand` terug.

Uitleg. `down` verwijdert wat wegwerpbaar is; de volumes, die de gegevens dragen, vereisen `-v`. Dat is het vangnet van Compose.

Stap 5 — Ontwikkeling en productie

```
podman compose down
podman compose -f compose.yaml -f compose.prod.yaml config | grep -E 'restart|APP_PROFILE|5432'
podman compose -f compose.yaml -f compose.prod.yaml up -d ; sleep 12
podman compose ps
podman inspect stack_api_1 --format '{{.HostConfig.RestartPolicy.Name}}'
podman exec stack_api_1 env | grep APP_PROFILE
podman compose exec api env | grep APP_PROFILE
podman compose -f compose.yaml -f compose.prod.yaml down
```

Observeer in `config`: drie keer `restart: unless-stopped`, `APP_PROFILE: prod`, en **geen** `5432` (de override wordt niet meer gelezen); in `ps`, `db` zonder gepubliceerde poort; `unless-stopped`; `APP_PROFILE=prod` via `podman exec`… maar `APP_PROFILE=dev` via `podman compose exec` zonder `-f`: dat heeft de override opnieuw gelezen.

Uitleg. `-f` is expliciet en exclusief; `exec` stelt de omgeving opnieuw samen op basis van de bestanden die je het geeft, niet op basis van de container. Geef altijd dezelfde `-f` (of `COMPOSE_FILE`), of gebruik `podman exec`.

```
API_TAG=2.0 podman compose config | grep 'image: localhost'
```

Observeer `image: localhost/api:2.0`: de substitutie vanuit de shell gaat voor op `.env`. Er bestaat geen image `2.0` — `config` controleert dat niet.

Stap 6 — Twee API's, en de grens van podman-compose

```
podman compose up -d --scale api=2 ; sleep 12
podman compose ps
podman run --rm --network stack_default alpine nslookup api 10.89.0.1 | grep Address
for i in 1 2 3 4 ; do curl -s -o /dev/null -w '%{http_code} '
localhost:18080/api/actuator/health ; done ; echo
```

Observeer `stack_api_1` en `stack_api_2`, allebei `(healthy)`; `nslookup` geeft, na de regel van de server, **twee** adressen voor `api` (`10.89.0.3` en `10.89.0.4`); vier keer `200`.

```
podman compose down -v 2>&1 | tail -2
podman ps -a --format '{{.Names}}'
podman rm -f -t 0 stack_api_2
podman compose down -v
podman ps -a --format '{{.Names}}' ; podman pod ls --format '{{.Name}}' ; podman volume ls --
format '{{.Name}}'
```

Observeer de mislukking van de eerste `down -v`: `cannot remove container … as it is running` en `"stack_default" has associated containers` — `stack_api_2` is gebleven, en `stack_db_1` met hem. Na het bij naam verwijderen van `stack_api_2` moppert de tweede `down -v` over wat al verdwenen is (`no container with name or ID "stack_web_1" found`) maar ruimt de rest op: geen container, geen pod, geen volume meer.

Uitleg. podman-compose beheert de instanties van `--scale` niet bij `down`. Docker Compose wel. Onthoud het herstel: `podman ps -a` lezen, bij naam verwijderen, `down` opnieuw starten.

Stap 7 — Bouwen vanuit Compose

```
podman compose build 2>&1 | tail -2
```

Observeer `Successfully tagged localhost/api:1.0`: de `build: context: ../05` heeft de Dockerfile van lab 05 opnieuw afgespeeld (uit de cache), en de image draagt de naam die `image:` declareert.

Uitleg. `build` + `image` in dezelfde dienst: Compose bouwt **en** benoemt. In CI volstaan `podman compose build` en `podman compose up -d` om de hele stack te testen.

Opruimen

```
podman compose down -v
podman ps -a --format '{{.Names}}' ; podman volume ls --format '{{.Name}}' ; podman network ls
--format '{{.Name}}'
```

Observeer dat alles leeg is (alleen het netwerk `podman` blijft). Hou `~/labo-docker/09/` volledig, `db_password.txt` inbegrepen, en de images `api:1.0`, `postgres:16-alpine`, `nginx:alpine`: lab 10 baat deze stack uit.

Wat je nu moet kunnen beweren

- `podman compose up -d` maakt netwerk, volume, pod en containers aan in de volgorde van `depends_on`: `db` healthy vóór `api`, `api` healthy vóór `web`.
- Alles draagt het voorvoegsel van het project: `stack_db_1`, `stack_default`, `stack_pgdata`, `pod_stack`.

- `config` toont de samenvoeging: automatische override, expliciete `-f` die ze negeert, `${API_TAG:-1.0}` gesubstitueerd vanuit `.env` of de shell.
- `down` bewaart `stack_pgdata` en de tabel `clients`; `down -v` verwijdert ze.
- `stop` / `start` van de API → nieuw IP → `502` bij nginx, `200` na `restart web`.
- `podman compose exec` zonder de juiste `-f` liegt (`dev`), `podman exec` zegt de waarheid (`prod`).
- `--scale api=2` geeft twee adressen aan de naam `api`; podman-compose kan ze niet verwijderen: `podman rm -f -t 0 stack_api_2` vóór `down`.